

Comparison of Long Short-Term Memory and Echo State Networks for Chaotic Time-Series Forecasting: The Mackey-Glass System

Project Report

Course Name: Neural Networks and Deep Learning

Module 1: Theoretical Foundations · Module 2: Advanced topics

Amal Nasharti

PhD in Physics and Astronomy
Università degli Studi di Firenze

Delivery Date: 7 September 2026

1 Motivation and Problem Definition

The choice of this project was motivated by my research interests in quantum machine learning (QML) [1], which explores the intersection of machine learning and quantum computing. Despite its potential, current QML approaches face important limitations on near-term quantum hardware [2]. In particular, predominant variational algorithms [3] rely on gradient estimation, which can place a significant computational burden on the limited resources of current quantum devices.

Reservoir computing (RC) provides an alternative machine learning paradigm that avoids costly gradient-based optimization [4]. It relies on a dynamical system, called the *reservoir*, whose internal connections remain fixed while only the output layer is trained. This paradigm has been extended to quantum systems through quantum reservoir computing (QRC) [5], where a quantum system replaces the classical reservoir. Understanding classical RC therefore represents a natural preliminary step toward investigating its quantum counterpart and constitutes the main motivation for this project.

Reservoir computing has been applied to a wide range of problems, including signal classification, time-series prediction, control of dynamical systems, and partial differential equation computation [6]. Among these applications, time-series forecasting is particularly relevant to the research interests of my group and therefore became the focus of this project. RC has shown particularly promising results in the prediction of chaotic dynamics, with the Mackey–Glass time series being a well-established benchmark [7, 8, 9]. At the same time, its application to real-world forecasting problems, including weather prediction [10, 11], remains more challenging.

As a first step toward the study of more complex real-world time series, the Mackey–Glass time series was selected as the benchmark for this project. It is generated by a nonlinear delay differential equation, defined as [12]

$$\frac{du(t)}{dt} = \frac{0.2 u(t - \tau)}{1 + u(t - \tau)^{10}} - 0.1 u(t), \quad (1)$$

where τ represents the time delay and plays a fundamental role in determining the dynamics of the system. As τ increases, the behaviour of the system becomes progressively more complex and, for sufficiently large values ($\tau > 16.8$), chaotic dynamics emerge. Although a chaotic system is governed by deterministic equations, its evolution is highly sensitive to small differences in the initial conditions. Consequently, initially similar trajectories can diverge significantly over time, making the prediction of future states a particularly challenging problem. In the experiments presented in this work, the delay parameter was set to $\tau = 17$, a value commonly adopted in the literature [12]. The time-series data were obtained from the GitHub repository in Ref. [13]. Figure 1 shows a representative segment of the time series used in the experiments.

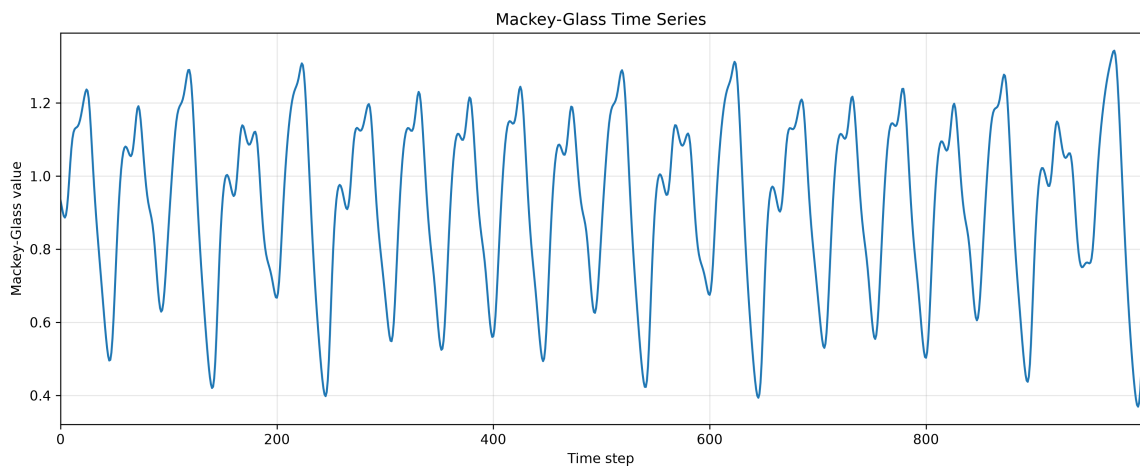


Figure 1: A portion of the Mackey–Glass timeseries with $\tau = 17$.

This project focuses on the Echo State Network (ESN) [7], one of the most widely used RC architectures, applied to the Mackey–Glass time series. Its performance is compared with that of a Long Short-Term

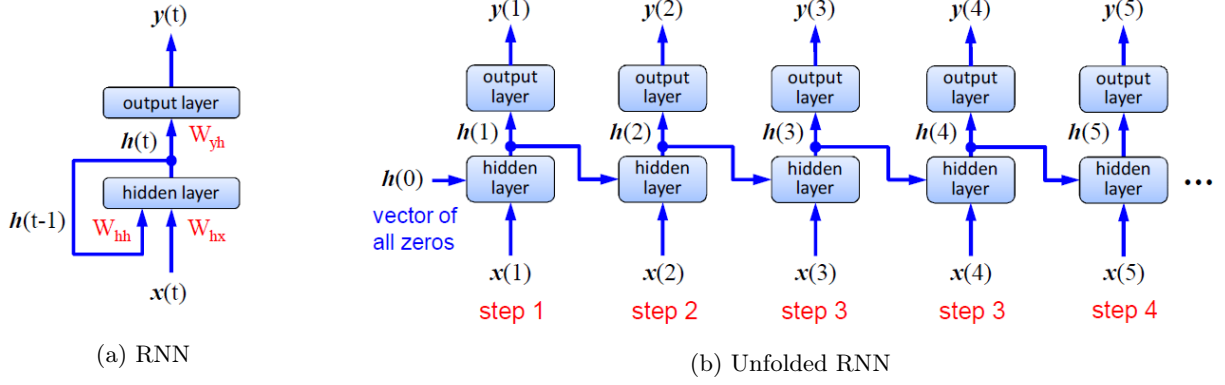


Figure 2: (a) Representation of a simple RNN, (b) Unfolded representation of an RNN.

Memory (LSTM) network [14] on the same one-step-ahead prediction task, considering both prediction error (MSE) and computational time. The software was developed in Python using the PyTorch framework.

The remainder of the report introduces the theoretical foundations of LSTM and ESN, describes the software implementation, and presents the experimental comparison of the two models. Finally, the main findings, limitations, and possible directions for future work are discussed.

2 Theoretical Background

Recurrent Neural Networks (RNNs) are designed to process sequential data by maintaining an internal state that retains information from previous time steps, enabling them to capture temporal dependencies. Among recurrent architectures, this work focuses on Long Short-Term Memory (LSTM) networks and Echo State Networks (ESNs), the latter belonging to the alternative Reservoir Computing (RC) paradigm.

2.1 Long Short-Term Memory Networks

LSTM networks [14] were developed to address the limitations of conventional recurrent neural networks in modelling long-term dependencies. As illustrated in Fig. 2(a), a simple RNN maintains a hidden state $h(t)$ that, at each time step t , is computed from both the current input $x(t)$ and the previous hidden state $h(t-1)$. This recurrent mechanism allows information from previous time steps to influence subsequent computations, while the same set of weights is shared throughout the sequence. The hidden state and output can be expressed as

$$h(t) = f(W_{hh}h(t-1) + W_{hx}x(t) + b_h), \quad (2)$$

$$y(t) = g(W_{yh}h(t) + b_y), \quad (3)$$

where W_{hx} , W_{hh} , and W_{yh} denote the input-to-hidden, hidden-to-hidden, and hidden-to-output weight matrices, respectively, while f and g represent the corresponding activation functions. Typical choices include tanh or ReLU for the hidden-state activation f , and sigmoid or softmax for the output activation g , depending on the task.

The recurrent structure can be unfolded over time, as shown in Fig. 2(b), and trained using Back-propagation Through Time (BPTT), whereby gradients are propagated backward across the sequence. Over long sequences, however, gradients may progressively decrease, leading to the *vanishing gradient* problem and limiting the ability of conventional RNNs to capture long-term dependencies.

LSTM networks address this limitation by replacing the simple recurrent unit with a more complex LSTM cell, illustrated in Fig. 3. The LSTM introduces a cell state $c(t)$ alongside the hidden state $h(t)$, while the information flow is regulated by three gating mechanisms: the forget, input, and output gates. Each gate uses a sigmoid activation function, producing values between 0 and 1 that determine how much information is allowed to pass.

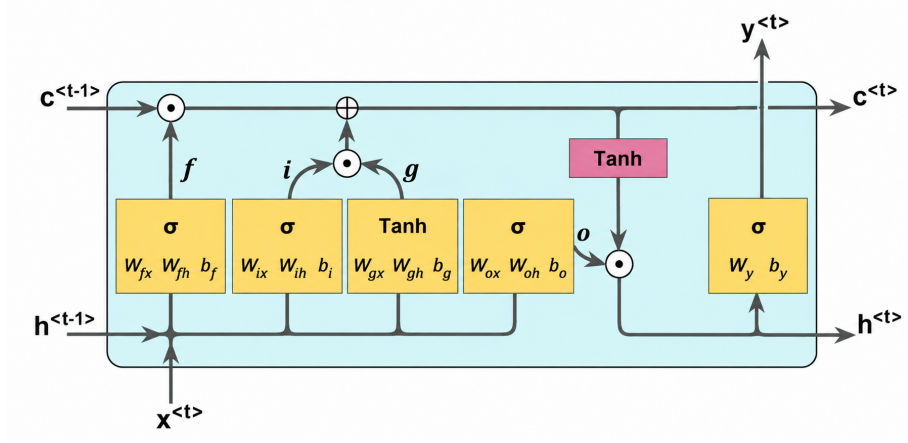


Figure 3: Representation of an LSTM cell.

The *forget gate* determines which information from the previous cell state should be retained:

$$\mathbf{f}(t) = \sigma(\mathbf{W}_{fx}\mathbf{x}(t) + \mathbf{W}_{fh}\mathbf{h}(t-1) + \mathbf{b}_f). \quad (4)$$

The *input gate* controls the addition of new information to the cell state. First, the gate activation $\mathbf{i}(t)$ determines how much new information should be accepted:

$$\mathbf{i}(t) = \sigma(\mathbf{W}_{ix}\mathbf{x}(t) + \mathbf{W}_{ih}\mathbf{h}(t-1) + \mathbf{b}_i). \quad (5)$$

At the same time, a candidate state $\mathbf{g}(t)$ is generated from the current input and the previous hidden state. It represents the new information that could be added to the cell state:

$$\mathbf{g}(t) = \tanh(\mathbf{W}_{gx}\mathbf{x}(t) + \mathbf{W}_{gh}\mathbf{h}(t-1) + \mathbf{b}_g). \quad (6)$$

The input gate $\mathbf{i}(t)$ therefore determines how much of the candidate state $\mathbf{g}(t)$ is incorporated into the cell state. The cell state is then updated by combining the retained information from the previous state with the selected new information:

$$\mathbf{c}(t) = \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{i}(t) \odot \mathbf{g}(t), \quad (7)$$

where $\odot$ denotes element-wise multiplication. The *output gate* determines which part of the updated cell state contributes to the hidden state:

$$\mathbf{o}(t) = \sigma(\mathbf{W}_{ox}\mathbf{x}(t) + \mathbf{W}_{oh}\mathbf{h}(t-1) + \mathbf{b}_o), \quad (8)$$

$$\mathbf{h}(t) = \mathbf{o}(t) \odot \tanh(\mathbf{c}(t)). \quad (9)$$

Finally, the network output $\mathbf{y}(t)$ is computed from the hidden state $\mathbf{h}(t)$ through the transformation:

$$\mathbf{y}(t) = \sigma(\mathbf{W}_y\mathbf{h}(t) + \mathbf{b}_y). \quad (10)$$

Thus, while $\mathbf{h}(t)$ is propagated to the next time step, $\mathbf{y}(t)$ represents the network output at the current time step. Overall, the combination of the cell state and gating mechanisms allows the LSTM to selectively retain, update, or discard information as the sequence is processed, improving its ability to capture long-term temporal dependencies compared with conventional RNNs.

2.2 Echo State Networks

ESN [7] consists of three main components: an input layer, a recurrent *reservoir*, and a *readout* layer, as illustrated in Fig. 4. Several formulations and notational conventions for ESNs can be found in the literature. In the following, the formulation adopted in Ref. [12] is used as reference.

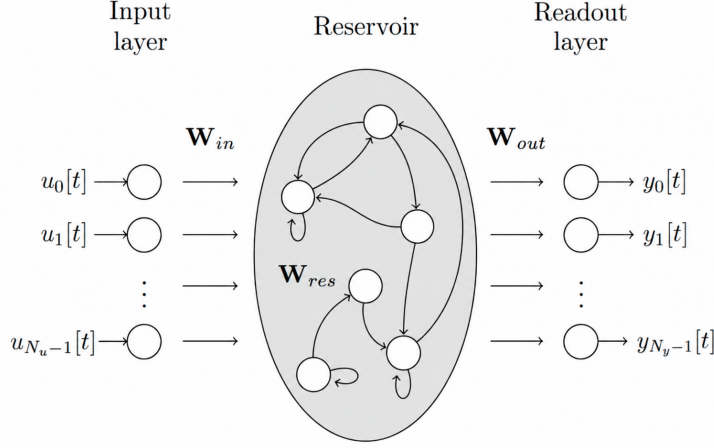


Figure 4: Schematic representation of the ESN architecture.

The reservoir consists of recurrent units whose weights are randomly initialized and remain fixed. At each time step, the reservoir state $\mathbf{x}(t)$ depends on the current input $\mathbf{u}(t)$ and on the previous reservoir state $\mathbf{x}(t-1)$:

$$\mathbf{x}(t) = f(\mathbf{W}_{in}\mathbf{u}(t) + \mathbf{W}_{res}\mathbf{x}(t-1)), \quad (11)$$

where $\mathbf{W}_{in}$ and $\mathbf{W}_{res}$ denote the input and reservoir weight matrices, respectively, and f is typically a tanh activation function. Through its recurrent dynamics, the reservoir transforms the input history into a rich internal representation encoded in its states $\mathbf{x}(t)$. These states are then used by the readout layer to generate the network output:

$$\mathbf{y}(t) = \mathbf{W}_{out}\mathbf{x}(t), \quad (12)$$

where $\mathbf{W}_{out}$ denotes the readout weights. Unlike the reservoir weights, which remain fixed, the readout weights are learned from the training data. Therefore, ESN training does not require optimization of the recurrent connections through backpropagation.

Before processing the input sequence, the reservoir and input weights must be initialized. To obtain sparse reservoir connectivity, each recurrent connection is assigned a random weight with probability p and set to zero otherwise. The parameter p , referred to as *reservoir connectivity*, therefore controls the fraction of active recurrent connections. Similarly, the input weights are randomly initialized according to

$$\mathbf{W}_{in} \sim \mathcal{U}(-\sigma_{in}, \sigma_{in}), \quad (13)$$

where $\mathcal{U}$ denotes a uniform distribution and σ_{in} is the *input scaling* parameter, which controls the magnitude of the input weights and thus the influence of the input on the reservoir dynamics.

A fundamental requirement of an ESN is the *echo state property* (ESP), according to which the reservoir state should be determined by the input history rather than by its initial condition. In other words, the influence of the initial state should progressively fade over time. The spectral radius ρ , defined as the largest absolute eigenvalue of $\mathbf{W}_{res}$, plays an important role in controlling this behaviour. Since the reservoir weights are randomly initialized, the matrix is commonly rescaled to obtain a desired spectral radius:

$$\mathbf{W}_{res} \leftarrow \frac{\rho}{\rho(\mathbf{W}_{res})} \mathbf{W}_{res}, \quad (14)$$

where $\rho(\mathbf{W}_{res})$ is the spectral radius of the initially generated matrix, and ρ is the desired value of the spectral radius. A spectral radius smaller than one is a commonly used sufficient, but not necessary, condition for the ESP.

During the initial time steps, the reservoir state may still be strongly influenced by its initial condition. For this reason, a *washout phase* is introduced: the reservoir is allowed to evolve for a predefined number of steps and the corresponding states are discarded before fitting the readout.

The standard reservoir update may also rapidly forget information associated with slowly varying signals. To mitigate this effect, *leaky integration* can be introduced by combining the previous reservoir state with the newly computed state:

$$\mathbf{x}(t) = \alpha \mathbf{x}(t-1) + (1-\alpha) f(\mathbf{W}_{in} \mathbf{u}(t) + \mathbf{W}_{res} \mathbf{x}(t-1)), \quad (15)$$

where α is the leaking parameter and controls the contribution of the previous reservoir state to the current state.

Once the reservoir states have been computed, only the readout weights need to be trained. In this work, they are determined using *ridge regression*, which minimizes the prediction error while penalizing large readout weights:

$$\min_{\mathbf{W}_{out}} \left\| \mathbf{W}_{out} \mathbf{X} - \hat{\mathbf{Y}} \right\|_2^2 + \lambda \left\| \mathbf{W}_{out} \right\|_2^2, \quad (16)$$

where $\mathbf{X}$ contains the reservoir states, $\hat{\mathbf{Y}}$ the corresponding target values, and $\lambda > 0$ is the ridge regularization coefficient. The corresponding solution is

$$\mathbf{W}_{out} = \hat{\mathbf{Y}} \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1}, \quad (17)$$

where $\mathbf{I}$ is the identity matrix. The readout weights can therefore be obtained directly, without iterative gradient-based training.

Typical values and practical guidelines for the selection of the main ESN hyperparameters can be found in the course material.

3 Software Implementation

The software developed for this project is provided as a ZIP file and is also available in the [mackey-glass-lstm-esn](#) repository. It is organized around three independent workflows: `main.py` runs the complete LSTM and ESN prediction pipeline, `fine_tuning.py` performs random-search hyperparameter optimization, and `app.py` launches the Gradio-based graphical interface. Their supporting files are contained in the `main/`, `fine_tuning/`, and `gui/` directories, respectively.

Reproducibility is ensured by explicitly controlling random seeds throughout the experimental workflows. Further details on the repository structure and execution instructions are provided in the `README.md` file.

3.1 Main Experimental Workflow

The `main.py` entry point executes the complete prediction pipeline for both LSTM and ESN. The required inputs are stored in the `main/input/` directory and include a CSV file containing the time series, together with a JSON configuration file defining the random seed, data-splitting parameters, and model hyperparameters. To ensure reproducibility of the experiments, the random seed is fixed to 0 and consistently applied to Python’s `random` module, NumPy, and PyTorch. When CUDA is available, the same seed is also applied to all CUDA devices. This ensures that the stochastic components controlled by these random number generators, including model initialization, are reproducible across repeated executions under the same experimental conditions.

For each architecture, the user can choose whether to train a new model or use previously trained parameters through the `LOAD_PRETRAINED_LSTM` and `LOAD_PRETRAINED_ESN` flags. When pretrained models are selected, the corresponding weights are loaded from the `main/weights/` directory.

The input time series is first chronologically divided into training, validation, and test sets according to the indices specified in the configuration file. The three subsets are then standardized using z-score normalization. To avoid information leakage, the mean and standard deviation are computed exclusively from the training set and subsequently applied to all three subsets:

$$x_{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}. \quad (18)$$

After preprocessing, the normalized data are processed independently by the LSTM and ESN pipelines, following model-specific procedures.

For the LSTM, each normalized subset is transformed into input-target pairs using a sliding-window approach. Given a sequence length L , each input consists of L consecutive observations,

$$[x(t - L + 1), \dots, x(t)], \quad (19)$$

while the following value $x(t + 1)$ is used as the target. The window is then shifted forward by one time step, converting the time series into a supervised one-step-ahead prediction problem. The resulting samples are organized into batches according to the specified `batch_size`.

The LSTM architecture developed in this work consists of a single LSTM layer followed by a fully connected linear layer. For each input sequence, the LSTM processes the observations sequentially and encodes their temporal information in the hidden state. The final hidden state, whose dimension is determined by the `hidden_size` parameter, is then passed to the linear layer, which maps this representation to a single scalar value corresponding to the one-step-ahead prediction.

When training a new LSTM, the optimization parameters are specified in the configuration file. The network is trained using the Adam optimizer to minimize the Mean Squared Error (MSE), while the validation loss is evaluated after each epoch. Early stopping is employed to prevent overfitting: if the validation loss does not improve for a predefined number of epochs, specified by `patience`, training is stopped and the weights associated with the best validation performance are saved in the `main/weights/` directory for subsequent use. When `LOAD_PRETRAINED_LSTM` is enabled, the training stage is skipped and the corresponding saved weights are loaded directly from this directory. Finally, the selected LSTM model generates predictions for the training, validation, and test sets. Training time and test-set inference time are also recorded for the subsequent comparison with the ESN.

For the ESN, each normalized subset is converted into one-step-ahead input-target pairs, where each value $x(t)$ is used to predict the immediately following observation $x(t + 1)$. Unlike the LSTM, no sliding window is explicitly constructed, since temporal information is progressively encoded in the reservoir state as the sequence is processed.

When training a new ESN, the network is initialized using the hyperparameters specified in the configuration file, following the procedure described in Section 2.2. The training sequence is then passed through the reservoir and, after discarding the initial washout states, the readout weights are fitted using ridge regression. The resulting model weights are saved in the `main/weights/` directory for subsequent use. When `LOAD_PRETRAINED_ESN` is enabled, the fitting stage is skipped and the previously saved weights are loaded directly. Once the readout weights have been fitted or loaded, the ESN is evaluated on the training, validation, and test sets. The computational cost is also measured by recording the fitting time and the inference time on the test set, enabling a direct comparison with the LSTM.

For both models, the predictions are transformed back to the original scale and evaluated by computing the MSE on the training, validation, and test sets. The resulting metrics, together with the prediction plots and computational times, are saved in the `main/output/` directory and used for the final comparison between the two approaches.

3.2 Hyperparameter Tuning

Hyperparameter tuning was performed using *random search*, which explores the hyperparameter space by sampling a predefined number of configurations from specified search ranges. Compared with an exhaustive grid search, this approach allows a broader exploration of the parameter space without evaluating every possible combination [12].

Separate search spaces were defined for the LSTM and ESN, as summarized in Table 1. Discrete parameters were sampled from predefined candidate values, whereas continuous parameters were sampled either uniformly or log-uniformly over the specified intervals. The search spaces were chosen to explore different levels of model complexity, temporal behaviour, and training conditions while maintaining a manageable computational cost. This choice also accounted for the available computational resources, as all experiments were performed on an AMD Ryzen 7 5700U CPU. The search-space definitions and the generated

Table 1: Hyperparameter search spaces and sampling strategies used in the random search.

Model	Parameter	Search space	Sampling
LSTM	Hidden size	{8, 16, 32, 64, 128}	Discrete
	Sequence length	{10, 20, 30, 50, 100}	Discrete
	Batch size	{16, 32, 64, 128}	Discrete
	Learning rate	$[10^{-4}, 10^{-2}]$	Log-uniform
	Number of epochs	{50, 100, 150, 200}	Discrete
	Patience	{5, 10, 15}	Discrete
ESN	Reservoir size	[50, 200]	Discrete uniform
	Spectral radius	[0.7, 0.99]	Uniform
	Reservoir connectivity	[0.01, 0.10]	Uniform
	Input scaling	[0.1, 1.0]	Uniform
	Washout	{50, 100, 200, 300, 500}	Discrete
	Leaking parameter	[0.1, 1.0]	Uniform
	Ridge coefficient	$[10^{-8}, 10^{-4}]$	Log-uniform

configurations used in the experiments are available in the `fine_tuning/input/search_spaces/` and `fine_tuning/input/configs/` directories, respectively.

Each configuration is fitted on the training set and evaluated on the validation set using MSE as the validation loss. Since model performance can depend on random initialization, each hyperparameter configuration is evaluated multiple times while varying only the random seed. This is particularly important for the ESN, whose randomly initialized input and reservoir weights can significantly affect the resulting reservoir dynamics and prediction performance. The mean and standard deviation of the validation loss are therefore computed across the different seeds for the same hyperparameter configuration, providing a more robust estimate of its performance and reducing the risk of selecting a configuration based on a particularly favorable or unfavorable initialization. The specific number and values of the random seeds used for each model are reported in Section 4.

The configuration achieving the lowest mean validation loss is selected as the best configuration found by the random search. The test set is not involved in hyperparameter selection and is reserved exclusively for the final evaluation of the selected models.

Since tuning procedures can require long execution times, the result of each trial is saved immediately after completion. This allows an interrupted search to resume without reevaluating previously completed trials. The complete results and the best configuration found are stored in the `fine_tuning/output/` directory.

The tuning procedure can be controlled directly from the entry-point script. For each model, `N_CONFIG_LSTM` and `N_CONFIG_ESN` specify the number of configurations to evaluate, while `SEEDS_LSTM` and `SEEDS_ESN` define the random seeds used for repeated evaluations. The `RESTART_LSTM` and `RESTART_ESN` flags determine whether the corresponding search starts from scratch or resumes from previously saved results.

3.3 Graphical User Interface

A graphical user interface (GUI) was developed using Gradio, an open-source Python framework for rapidly building web-based interfaces for machine learning applications. Gradio was selected because it can be directly integrated with Python functions, allowing the functionalities already implemented in the project to be exposed through an interactive interface without developing a separate front-end application or exposing the back-end via API. The interface can be launched by executing the `app.py` entry-point script. Once the application starts, Gradio provides a local URL in the terminal; opening this URL in a web browser gives access to the graphical interface and its functionalities.

Through the GUI, the user can upload and inspect a time-series dataset, define the training, validation, and test split, and visualize the input series. For both LSTM and ESN, it is possible either to train a new model by specifying its hyperparameters or to load a pretrained model together with its

Table 2: LSTM hyperparameter configuration selected by random search.

Parameter	Value
Hidden size	32
Sequence length	100
Batch size	16
Learning rate	2.29386×10^{-3}
Number of epochs	50
Patience	15

Table 3: ESN hyperparameter configuration selected by random search.

Parameter	Value
Reservoir size	61
Spectral radius	0.79407
Reservoir connectivity	0.01505
Input scaling	0.42262
Washout	300
Leaking parameter	0.23101
Ridge coefficient	1.33249×10^{-6}

configuration. The interface then displays the prediction results, evaluation metrics, and computational times.

The GUI also provides access to random-search hyperparameter tuning for both architectures, allowing the user to specify the number of configurations and random seeds used in the search. The best configurations obtained can be downloaded and subsequently used for model training. Model configurations and trained weights can also be exported for later reuse.

4 Results and Discussion

The random-search procedure described in the previous section was performed independently for the two architectures. For the LSTM, 50 hyperparameter configurations were evaluated over 5 different random seeds, corresponding to the integer values from 0 to 4. For the ESN, the search considered 150 configurations, each evaluated over 10 random seeds, from 0 to 9. This difference was motivated by the substantially different computational costs of the two models. On the AMD Ryzen 7 5700U CPU used for the experiments, the complete LSTM tuning procedure required approximately 4.5 hours, while the larger ESN search was completed in approximately 10 minutes. The lower computational cost of the ESN therefore allowed a larger number of configurations and random initializations to be explored.

For both models, the configuration achieving the lowest mean validation MSE across the different runs was selected. The resulting LSTM and ESN configurations are summarized in Tables 2 and 3, respectively. For readability, numerical hyperparameter values are rounded to five decimal places, with scientific notation used for small values. Full-precision values are available in the tuning results provided with the repository.

The hyperparameter configurations selected through random search were then used in the main experimental workflow. Both models were evaluated under the same data-splitting and preprocessing conditions, and the predictions were transformed back to the original scale of the Mackey–Glass time series before computing the MSE. The resulting prediction errors and computational times are reported in Table 4.

For the LSTM, Fig. 5 shows the evolution of the training and validation losses over the training epochs. Both losses decrease while remaining closely aligned, with no substantial divergence between the two curves, suggesting that the model does not exhibit evident signs of overfitting. A corresponding learning curve is not available for the ESN because its training procedure is fundamentally different: the reservoir weights remain fixed and the readout weights are determined directly through ridge regression, without

Table 4: Prediction performance and computational times of the LSTM and ESN models. MSE values are computed on the original scale of the Mackey–Glass time series.

Metric	LSTM	ESN
Training MSE	1.41036×10^{-7}	9.73523×10^{-6}
Validation MSE	1.32989×10^{-7}	1.70482×10^{-5}
Test MSE	1.44786×10^{-7}	1.84143×10^{-5}
Training/Fitting time (s)	82.041	0.311
Test inference time (s)	0.205	0.153

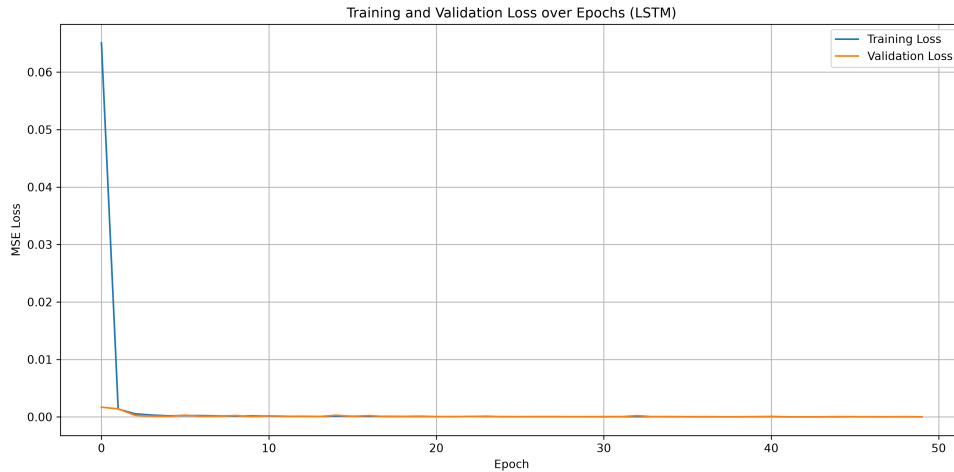


Figure 5: Training and validation MSE of the LSTM as a function of the training epoch.

iterative backpropagation over multiple epochs.

The results show a clear trade-off between predictive accuracy and computational cost. On the test set, the LSTM achieves an MSE of approximately 1.45×10^{-7} , compared with 1.84×10^{-5} for the ESN, indicating substantially higher prediction accuracy for the LSTM in the considered experiment. Moreover, the LSTM exhibits very similar training, validation, and test errors, suggesting good generalization to unseen data.

The ESN, however, provides a substantial advantage in terms of fitting time, requiring approximately 0.31 s compared with 82.04 s for the LSTM. Thus, the LSTM training procedure is approximately 264 times slower than the ESN fitting procedure. The difference in inference time is considerably smaller, with 0.205 s for the LSTM and 0.153 s for the ESN. Therefore, the main computational advantage of the ESN is observed during model fitting rather than during inference.

The prediction performance on the test set is also illustrated in Figs. 6 and 7, where the predictions of the LSTM and ESN are compared with the actual Mackey–Glass values. Both models successfully reproduce the temporal evolution of the series; however, the LSTM predictions show closer agreement with the target values, with the difference being noticeable at the beginning of the test set. For the ESN, this initial discrepancy is likely related to the reservoir state being initialized to zero at the beginning of the test sequence, resulting in an initial transient before the reservoir dynamics reflect the recent input history.

5 Conclusions and Future Work

This project provided a complete comparison between LSTM and ESN for one-step-ahead forecasting of the chaotic Mackey–Glass time series, covering model development, hyperparameter optimization, performance evaluation, and the implementation of an interactive graphical interface. Both architectures were able to reproduce the dynamics of the series, although with a clear trade-off between predictive accuracy

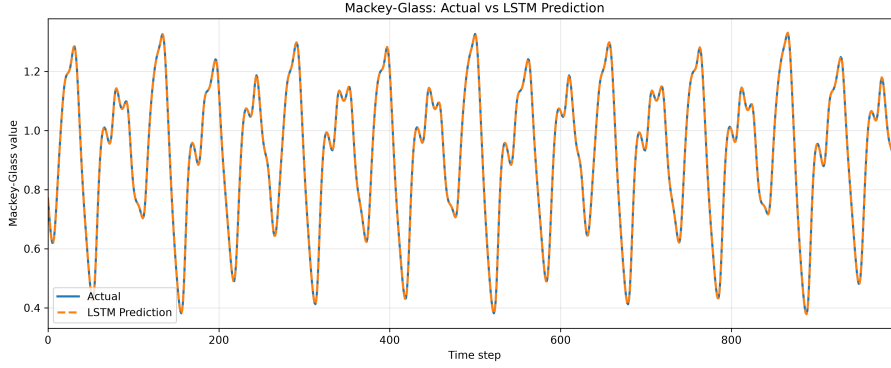


Figure 6: LSTM prediction on the test set. Only a portion of the test set is displayed for readability.

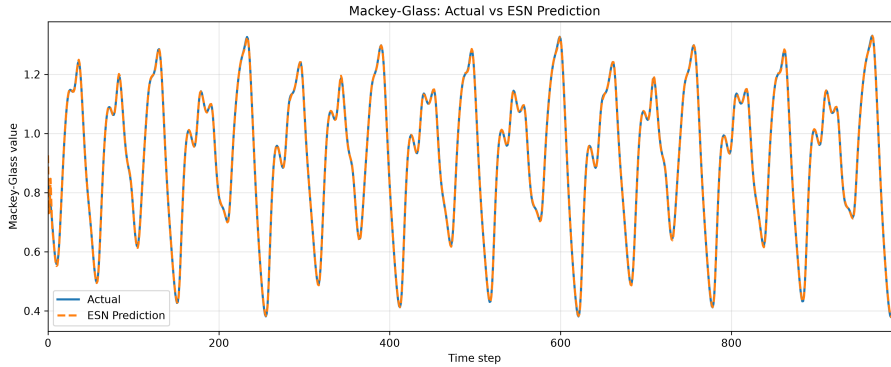


Figure 7: ESN prediction on the test set. Only a portion of the test set is displayed for readability.

and computational cost. The LSTM achieved the lowest prediction error, whereas the ESN required only a fraction of the fitting time, highlighting the computational efficiency of the reservoir computing approach.

The results also indicate that the current ESN implementation still offers significant room for improvement. While the LSTM test error is comparable to that reported in the reference work [12], the ESN test error obtained in this project is several orders of magnitude higher than the value reported therein, although the model still achieves a relatively low test error. This difference motivates a more detailed investigation of both the implementation and the experimental setup. In particular, the initialization of the reservoir state should be examined more carefully, as the transient observed at the beginning of the test sequence suggests that the current zero-state initialization may affect the initial predictions. A direct comparison with the ESN implementation available in the reference repository [13] would also help identify differences in architecture, state initialization, preprocessing, or evaluation procedures, providing useful indications on how the developed ESN could be further improved.

Future work should also investigate the generalization capabilities of both models beyond the Mackey-Glass benchmark, first considering other chaotic systems and subsequently more complex real-world time series. A more controlled comparison between LSTM and ESN could also be performed by considering architectures with comparable numbers of parameters, allowing the relationship between model complexity, predictive accuracy, and computational cost to be investigated more systematically.

Finally, the experimental framework itself could be extended. GPU acceleration could be introduced to assess how hardware acceleration affects the computational comparison between the two architectures, particularly for the more expensive LSTM training procedure. Moreover, an automated experiment-management system could be developed to store configurations, random seeds, metrics, computational times, and model weights across multiple runs, facilitating larger and more systematic experimental studies.

Overall, the present work establishes a complete and reproducible baseline for the comparison of LSTM

and ESN on chaotic time-series forecasting. The developed software and the results obtained provide a solid foundation for further optimization of the ESN architecture and for extending the analysis toward more challenging forecasting problems.

A Additional LSTM Training Curves

To further illustrate the training behaviour of the LSTM, Figs. 8 and 9 show the training and validation loss curves obtained for the first two hyperparameter configurations evaluated during the random search. The corresponding hyperparameters are reported in Table 5. For readability, decimal values are rounded to five decimal places. The curves show no evident sign of overfitting.

Table 5: LSTM hyperparameter configurations used for the additional training curves. Best validation loss is also shown.

Parameter	Trial 1	Trial 2
Hidden size	8	8
Sequence length	10	100
Batch size	64	16
Learning rate	3.08876×10^{-4}	1.51700×10^{-3}
Number of epochs	100	50
Patience	15	5
Best validation loss	3.66101×10^{-5}	1.94792×10^{-6}

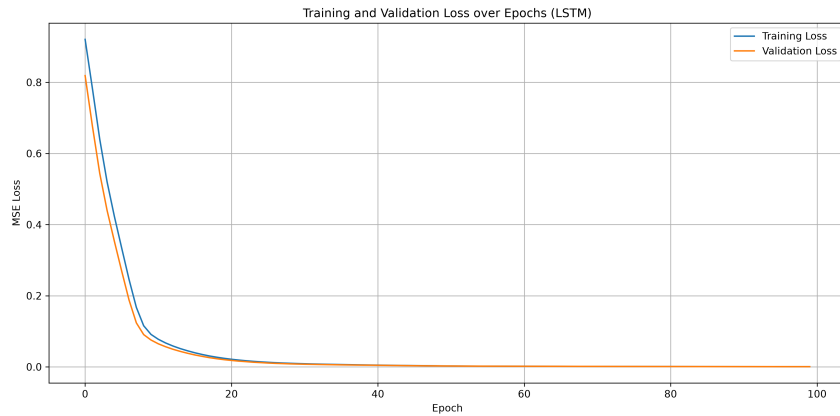


Figure 8: Training and validation MSE for the LSTM configuration corresponding to Trial 1 in Table 5.



Figure 9: Training and validation MSE for the LSTM configuration corresponding to Trial 2 in Table 5.

References

- [1] Jacob Biamonte et al. “Quantum machine learning”. In: *Nature* 549.7671 (2017), pp. 195–202.
- [2] John Preskill. “Quantum computing in the NISQ era and beyond”. In: *Quantum* 2 (2018), p. 79.
- [3] Marco Cerezo et al. “Variational quantum algorithms”. In: *Nature Reviews Physics* 3.9 (2021), pp. 625–644.
- [4] Herbert Jaeger and Harald Haas. “Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication”. In: *science* 304.5667 (2004), pp. 78–80.
- [5] Milan Kornjača et al. “Large-scale quantum reservoir learning with an analog quantum computer”. In: *arXiv preprint arXiv:2407.02553* (2024).
- [6] Min Yan et al. “Emerging opportunities and challenges for the future of reservoir computing”. In: *Nature Communications* 15.1 (2024), p. 2056.
- [7] Herbert Jaeger. “The “echo state” approach to analysing and training recurrent neural networks-with an erratum note”. In: *Bonn, Germany: German national research center for information technology gmd technical report* 148.34 (2001), p. 13.
- [8] Jonathan Dong et al. “Optical reservoir computing using multiple light scattering for chaotic systems prediction”. In: *IEEE Journal of Selected Topics in Quantum Electronics* 26.1 (2019), pp. 1–12.
- [9] John Moon et al. “Temporal data classification and forecasting using a memristor-based reservoir computing system”. In: *Nature Electronics* 2.10 (2019), pp. 480–487.
- [10] Troy Arcomano et al. “A hybrid atmospheric model incorporating machine learning can capture dynamical processes not captured by its physics-based component”. In: *Geophysical Research Letters* 50.8 (2023), e2022GL102649.
- [11] Troy Arcomano et al. “A machine learning-based global atmospheric forecast model”. In: *Geophysical Research Letters* 47.9 (2020), e2020GL087776.
- [12] Stefano Nardo. “An empirical comparison of recurrent neural networks on sequence modeling”. MA thesis. Università di Pisa, 2019.
- [13] Stefano Nardo. *PyTorch-ESN: Echo State Networks in PyTorch*. <https://github.com/stefanonardo/pytorch-esn>. 2018.
- [14] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural computation* 9.8 (1997), pp. 1735–1780.